Week 11 – Database Automation using Views and Procedures

Scenario:

The e-commerce company wants to simplify database management by automating customer order tracking, payment verification, product stock updates, and sales reporting. To reduce manual SQL operations and improve data accuracy, the company plans to implement Views, Stored Procedures, and Functions.

Students will develop database automation features in the existing E-Commerce Database System to improve efficiency, minimize repetitive queries, and simplify business operations.

1.Create Views for Frequently Used Reports

Concept:

A View is a virtual table created from one or more SQL queries. It stores the query definition and provides simplified access to frequently used information

1.Customer Order Report View

- Customer Name
- Order ID
- Order Date
- Total Amount
- Order Status

```
SQL> CREATE VIEW Customer_Order_Report AS SELECT C.CUSTOMER_NAME, O.ORDER_ID, O.ORDER_DATE, O.TOTAL_AMOUNT, O.ORDER_STATUS F
ROM CUSTOMER C JOIN ORDERS O ON C.CUSTOMER_ID = O.CUSTOMER_ID;

View created.

SQL> SELECT * FROM Customer_Order_Report;

CUSTOMER_NAME                                     ORDER_ID ORDER_DAT
------------------------------------------------- ---------- ---------
TOTAL_AMOUNT ORDER_STAT
------------ ----------
Arun                                                  1001 27-SEP-26
       67500 Pending

Priya                                                 1002 27-SEP-26
       25000 Shipped

Kumar                                                 1003 27-SEP-26
        1500 Delivered


CUSTOMER_NAME                                     ORDER_ID ORDER_DAT
------------------------------------------------- ---------- ---------
TOTAL_AMOUNT ORDER_STAT
------------ ----------
Divya                                                 1004 27-SEP-26
        4200 Pending

Rahul                                                 1005 27-SEP-26
        8000 Delivered
```

2. Product Sales Report View

Display:

- Product Name
- Quantity Sold
- Total Revenue

```
SQL> CREATE VIEW Product_Sales_Report AS SELECT P.PRODUCT_NAME, SUM(OD.QUANTITY) AS QUANTITY_SOLD, SUM(OD.QUANTITY * OD.PRIC
E) AS TOTAL_REVENUE FROM PRODUCT P JOIN ORDER_DETAILS OD ON P.PRODUCT_ID = OD.PRODUCT_ID GROUP BY P.PRODUCT_ID, P.PRODUCT_NA
ME;

View created.

SQL> SELECT * FROM Product_Sales_Report;

PRODUCT_NAME         QUANTITY_SOLD TOTAL_REVENUE
-------------------- ------------- -------------
Laptops                          1         67500
Mobile                           1         25000
TShirt                           2          1500
Shoes                            2          4200
Keyboard                         1          8000
```

3. Payment Status View

Display:

- Order ID
- Payment Mode
- Payment Status
- Transaction Amount

```
SQL> CREATE VIEW Payment_Status_Report AS SELECT ORDER_ID, PAYMENT_MODE, PAYMENT_STATUS, TRANSACTION_AMOUNT FROM PAYMENT;

View created.

SQL> SELECT * FROM Payment_Status_Report;

  ORDER_ID PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
---------- -------------------- ---------- ------------------
      1001 UPI                  Success                67500
      1002 Card                 Success                25000
      1003 Cash                 Success                 1500
      1004 Net Banking          Pending                 4200
      1005 UPI                  Success                 8000
      1001 Card                 Failed                  1000
      1002 UPI                  Pending                 5000
      1003 Net Banking          Failed                  2000

8 rows selected.
```

2.Develop Stored Procedures for Order Processing Concept:

A Stored Procedure is a collection of SQL statements stored in the database that can be executed whenever required. It helps automate repeated operations

1.Add New Customer Order

Process:

- Insert order details.
- Add purchased products.
- Calculate total amount.

```
SQL> CREATE OR REPLACE PROCEDURE ADD_CUSTOMER_ORDER (
  2  P_ORDER_ID NUMBER,
  3  P_CUSTOMER_ID NUMBER,
  4  P_PRODUCT_ID VARCHAR2,
  5  P_QUANTITY NUMBER
  6  )
  7  AS
  8  V_PRICE NUMBER(10,2);
  9  V_TOTAL NUMBER(10,2);
 10  BEGIN
 11  SELECT PRICE INTO V_PRICE FROM PRODUCT WHERE PRODUCT_ID = P_PRODUCT_ID;
 12  V_TOTAL := V_PRICE * P_QUANTITY;
 13  INSERT INTO ORDERS VALUES (P_ORDER_ID, SYSDATE, V_TOTAL, 'Pending', P_CUSTOMER_ID);
 14  INSERT INTO ORDER_DETAILS VALUES ('OD'||P_ORDER_ID, P_ORDER_ID, P_PRODUCT_ID, P_QUANTITY, V_PRICE);
 15  UPDATE PRODUCT SET STOCK_QUANTITY = STOCK_QUANTITY - P_QUANTITY WHERE PRODUCT_ID = P_PRODUCT_ID;
 16  COMMIT;
 17  END;
 18  /

Procedure created.

SQL> EXEC ADD_CUSTOMER_ORDER(1006, 1001, 'P1001', 2);

PL/SQL procedure successfully completed.

SQL> SELECT * FROM ORDERS WHERE ORDER_ID = 1006;

  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STAT CUSTOMER_ID
---------- --------- ------------ ---------- -----------
      1006 07-NOV-26       135000 Pending           1001

SQL> SELECT * FROM ORDER_DETAILS WHERE ORDER_ID = 1006;

ORDER_D   ORDER_ID PRODUCT   QUANTITY      PRICE
------- ---------- ------- ---------- ----------
OD1006        1006 P1001            2      67500
```

2. Update Order Status

Example:

Change:

Pending → Shipped → Delivered

```
SQL> CREATE OR REPLACE PROCEDURE UPDATE_ORDER_STATUS (P_ORDER_ID NUMBER, P_STATUS VARCHAR2)
  2  AS
  3  BEGIN
  4  UPDATE ORDERS SET ORDER_STATUS = P_STATUS WHERE ORDER_ID = P_ORDER_ID;
  5  COMMIT;
  6  END;
  7  /

Procedure created.

SQL> EXEC UPDATE_ORDER_STATUS(1001, 'Shipped');

PL/SQL procedure successfully completed.

SQL> EXEC UPDATE_ORDER_STATUS(1001, 'Delivered');

PL/SQL procedure successfully completed.

SQL> SELECT ORDER_ID, ORDER_STATUS FROM ORDERS;

  ORDER_ID ORDER_STAT
---------- ----------
      1006 Pending
      1001 Delivered
      1002 Shipped
      1003 Delivered
      1004 Pending
      1005 Delivered

6 rows selected.
```

3. Generate Customer Purchase History

Input: Customer_ID

Output:

- Order details
- Purchase amount
- Order status

```
SQL> CREATE OR REPLACE PROCEDURE GetCustomerOrders (P_CUST_ID NUMBER)
  2  AS
  3  BEGIN
  4  SELECT ORDER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS FROM ORDERS WHERE CUSTOMER_ID = P_CUST_ID;
  5  END;
  6  /

Warning: Procedure created with compilation errors.

SQL> CREATE OR REPLACE PROCEDURE GetCustomerOrders (P_CUST_ID NUMBER, P_RESULT OUT SYS_REFCURSOR)
  2  AS
  3  BEGIN
  4  OPEN P_RESULT FOR SELECT ORDER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS FROM ORDERS WHERE CUSTOMER_ID = P_CUST_ID;
  5  END;
  6  /

Procedure created.

SQL> VARIABLE R REFCURSOR;
SQL> EXEC GetCustomerOrders(1001, :R);

PL/SQL procedure successfully completed.

SQL> PRINT R;

  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STAT
---------- --------- ------------ ----------
      1006 07-NOV-26       135000 Pending
      1001 27-SEP-26        67500 Delivered
```

3. Create Functions for Sales Calculations

Concept: A Function performs a specific calculation and returns a value

1.Calculate Total Sales

Input:

- Date range

Output:

- Total revenue generated

```
SQL> CREATE OR REPLACE FUNCTION Calculate_Total_Sales (P_START_DATE DATE, P_END_DATE DATE)
  2  RETURN NUMBER
  3  AS
  4  V_TOTAL NUMBER(10,2);
  5  BEGIN
  6  SELECT NVL(SUM(TOTAL_AMOUNT),0) INTO V_TOTAL FROM ORDERS WHERE ORDER_DATE >= P_START_DATE AND ORDER_DATE < P
_END_DATE + 1;
  7  RETURN V_TOTAL;
  8  END;
  9  /

Function created.

SQL> SELECT Calculate_Total_Sales(TO_DATE('01-11-2026','DD-MM-YYYY'), TO_DATE('30-11-2026','DD-MM-YYYY')) AS TOTA
L_SALES FROM DUAL;

TOTAL_SALES
-----------
     135000
```

2. Calculate Customer Total Spending

Input:

- Customer_ID

Output:

- Total amount spent

```
SQL> CREATE OR REPLACE FUNCTION Calculate_Customer_Spending (P_CUSTOMER_ID NUMBER)
  2  RETURN NUMBER
  3  AS
  4  V_TOTAL NUMBER(10,2);
  5  BEGIN
  6  SELECT NVL(SUM(TOTAL_AMOUNT),0) INTO V_TOTAL FROM ORDERS WHERE CUSTOMER_ID = P_CUSTOMER_ID;
  7  RETURN V_TOTAL;
  8  END;
  9  /

Function created.

SQL> SELECT Calculate_Customer_Spending(1001) AS TOTAL_SPENDING FROM DUAL;

TOTAL_SPENDING
--------------
        202500
```

3. Calculate Product Revenue

Input:

- Product_ID

Output:

- Total sales amount

```
SQL> CREATE OR REPLACE FUNCTION Calculate_Product_Revenue (P_PRODUCT_ID VARCHAR2)
  2  RETURN NUMBER
  3  AS
  4  V_REVENUE NUMBER(10,2);
  5  BEGIN
  6  SELECT NVL(SUM(QUANTITY * PRICE),0) INTO V_REVENUE FROM ORDER_DETAILS WHERE PRODUCT_ID = P_PRODUCT_ID;
  7  RETURN V_REVENUE;
  8  END;
  9  /

Function created.

SQL> SELECT Calculate_Product_Revenue('P1001') AS TOTAL_REVENUE FROM DUAL;

TOTAL_REVENUE
-------------
       202500
```

4. Automate Repetitive Database Operations

Tasks: Implement automation for:

Order Processing Automation

When a customer places an order:

1. Insert order details.
2. Reduce product stock.
3. Create payment record.
4. Update order status.

```
SQL> CREATE OR REPLACE PROCEDURE PROCESS_ORDER (
  2  P_ORDER_ID NUMBER,
  3  P_CUSTOMER_ID NUMBER,
  4  P_PRODUCT_ID VARCHAR2,
  5  P_QUANTITY NUMBER,
  6  P_PAYMENT_MODE VARCHAR2
  7  )
  8  AS
  9  V_PRICE NUMBER(10,2);
 10  V_TOTAL NUMBER(10,2);
 11  V_STOCK NUMBER;
 12  V_PAYMENT_ID NUMBER;
 13  BEGIN
 14  SELECT PRICE, STOCK_QUANTITY INTO V_PRICE, V_STOCK FROM PRODUCT WHERE PRODUCT_ID = P_PRODUCT_ID FOR UPDATE;
 15
 16  IF V_STOCK < P_QUANTITY OR P_QUANTITY <= 0 THEN
 17  RAISE_APPLICATION_ERROR(-20001, 'Insufficient stock or invalid quantity');
 18  END IF;
 19
 20  V_TOTAL := V_PRICE * P_QUANTITY;
 21
 22  INSERT INTO ORDERS VALUES (P_ORDER_ID, SYSDATE, V_TOTAL, 'Shipped', P_CUSTOMER_ID);
 23
 24  INSERT INTO ORDER_DETAILS VALUES ('OD'||P_ORDER_ID, P_ORDER_ID, P_PRODUCT_ID, P_QUANTITY, V_PRICE);
 25
 26  UPDATE PRODUCT SET STOCK_QUANTITY = STOCK_QUANTITY - P_QUANTITY WHERE PRODUCT_ID = P_PRODUCT_ID;
 27
 28  SELECT NVL(MAX(PAYMENT_ID),0)+1 INTO V_PAYMENT_ID FROM PAYMENT;
 29
 30  INSERT INTO PAYMENT VALUES (V_PAYMENT_ID, P_ORDER_ID, SYSDATE, P_PAYMENT_MODE, 'Success', V_TOTAL);
 31
 32  COMMIT;
 33  EXCEPTION
 34  WHEN OTHERS THEN
 35  ROLLBACK;
 36  RAISE;
 37  END;
 38  /

Procedure created.
```

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, STOCK_QUANTITY FROM PRODUCT WHERE PRODUCT_ID = 'P1001';

PRODUCT_ID PRODUCT_NAME         STOCK_QUANTITY
---------- -------------------- --------------
P1001      Laptops                           0

SQL> SELECT PRODUCT_ID, PRODUCT_NAME, STOCK_QUANTITY FROM PRODUCT WHERE STOCK_QUANTITY > 0;

PRODUCT_ID PRODUCT_NAME         STOCK_QUANTITY
---------- -------------------- --------------
P1002      Mobile                           10
P1003      TShirt                           20
P1004      Shoes                            15
P1005      Keyboard                         10
P1006      Mouse                            15

SQL> EXEC PROCESS_ORDER(1007, 1001, 'P1002', 1, 'UPI');

PL/SQL procedure successfully completed.

SQL> SELECT * FROM ORDERS WHERE ORDER_ID = 1007;

  ORDER_ID ORDER_DAT TOTAL_AMOUNT ORDER_STAT CUSTOMER_ID
---------- --------- ------------ ---------- -----------
      1007 07-NOV-26        25000 Shipped           1001

SQL> SELECT * FROM PAYMENT WHERE ORDER_ID = 1007;

PAYMENT_ID   ORDER_ID PAYMENT_D PAYMENT_MODE         PAYMENT_ST
---------- ---------- --------- -------------------- ----------
TRANSACTION_AMOUNT
------------------
         9       1007 07-NOV-26 UPI                  Success
             25000
```

Inventory Update Automation

Automatically:

- Increase stock after new supply.
- Reduce stock after purchase.
- Identify low-stock products.

```
SQL> CREATE OR REPLACE PROCEDURE INCREASE_STOCK (
  2       P_PRODUCT_ID VARCHAR2,
  3       P_QUANTITY NUMBER
  4   )
  5   AS
  6   BEGIN
  7       UPDATE PRODUCT
  8       SET STOCK_QUANTITY = STOCK_QUANTITY + P_QUANTITY
  9       WHERE PRODUCT_ID = P_PRODUCT_ID;
 10
 11       COMMIT;
 12   END;
 13   /

Procedure created.

SQL> EXEC INCREASE_STOCK('P1002', 10);

PL/SQL procedure successfully completed.
```

```
SQL> CREATE OR REPLACE PROCEDURE REDUCE_STOCK (
  2       P_PRODUCT_ID VARCHAR2,
  3       P_QUANTITY NUMBER
  4   )
  5   AS
  6   BEGIN
  7       UPDATE PRODUCT
  8       SET STOCK_QUANTITY = STOCK_QUANTITY - P_QUANTITY
  9       WHERE PRODUCT_ID = P_PRODUCT_ID
 10       AND STOCK_QUANTITY >= P_QUANTITY;
 11
 12       IF SQL%ROWCOUNT = 0 THEN
 13           RAISE_APPLICATION_ERROR(-20001, 'Product not found or insufficient stock');
 14       END IF;
 15
 16       COMMIT;
 17   END;
 18   /

Procedure created.

SQL> EXEC REDUCE_STOCK('P1002', 2);

PL/SQL procedure successfully completed.
```

```
SQL> SELECT PRODUCT_ID, PRODUCT_NAME, STOCK_QUANTITY
  2  FROM PRODUCT
  3  WHERE STOCK_QUANTITY <= 5;

PRODUCT_ID PRODUCT_NAME         STOCK_QUANTITY
---------- -------------------- --------------
P1001      Laptops                           0

SQL> SELECT PRODUCT_ID, PRODUCT_NAME, STOCK_QUANTITY
  2  FROM PRODUCT
  3  ORDER BY PRODUCT_ID;

PRODUCT_ID PRODUCT_NAME         STOCK_QUANTITY
---------- -------------------- --------------
P1001      Laptops                           0
P1002      Mobile                           37
P1003      TShirt                           18
P1004      Shoes                            15
P1005      Keyboard                         10
P1006      Mouse                            15

6 rows selected.
```

Report Generation Automation

Automatically generate:

- Daily sales report.

```
SQL> CREATE OR REPLACE PROCEDURE DAILY_SALES_REPORT
  2  (P_DATE DATE)
  3  AS
  4      V_TOTAL_ORDERS NUMBER;
  5      V_TOTAL_SALES NUMBER;
  6  BEGIN
  7      DBMS_OUTPUT.PUT_LINE('DAILY SALES REPORT');
  8      DBMS_OUTPUT.PUT_LINE('----------------------------');
  9
 10      FOR R IN (
 11          SELECT ORDER_ID, ORDER_DATE, TOTAL_AMOUNT, ORDER_STATUS
 12          FROM ORDERS
 13          WHERE TRUNC(ORDER_DATE) = TRUNC(P_DATE)
 14      )
 15      LOOP
 16          DBMS_OUTPUT.PUT_LINE(
 17              'Order ID: ' || R.ORDER_ID ||
 18              ' Amount: ' || R.TOTAL_AMOUNT ||
 19              ' Status: ' || R.ORDER_STATUS
 20          );
 21      END LOOP;
 22
 23      SELECT COUNT(*), NVL(SUM(TOTAL_AMOUNT), 0)
 24      INTO V_TOTAL_ORDERS, V_TOTAL_SALES
 25      FROM ORDERS
 26      WHERE TRUNC(ORDER_DATE) = TRUNC(P_DATE);
 27
 28      DBMS_OUTPUT.PUT_LINE('Total Orders: ' || V_TOTAL_ORDERS);
 29      DBMS_OUTPUT.PUT_LINE('Total Sales: ' || V_TOTAL_SALES);
 30
 31  END;
 32  /

Procedure created.
```

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC DAILY_SALES_REPORT(DATE '2026-11-07');
DAILY SALES REPORT
---------------------------
Order ID: 1006 Amount: 135000 Status: Pending
Order ID: 1007 Amount: 25000 Status: Shipped
Order ID: 1008 Amount: 1600 Status: Shipped
Total Orders: 3
Total Sales: 161600

PL/SQL procedure successfully completed.

SQL> SHOW ERRORS PROCEDURE DAILY_SALES_REPORT;
No errors.
```

- Monthly revenue report.

```
SQL> CREATE OR REPLACE PROCEDURE MONTHLY_REVENUE_REPORT
  2  (P_YEAR NUMBER)
  3  AS
  4  BEGIN
  5      DBMS_OUTPUT.PUT_LINE('MONTHLY REVENUE REPORT');
  6      DBMS_OUTPUT.PUT_LINE('---------------------------');
  7
  8      FOR R IN (
  9          SELECT EXTRACT(MONTH FROM ORDER_DATE) AS MONTH_NO,
 10                 COUNT(ORDER_ID) AS TOTAL_ORDERS,
 11                 SUM(TOTAL_AMOUNT) AS TOTAL_REVENUE
 12          FROM ORDERS
 13          WHERE EXTRACT(YEAR FROM ORDER_DATE) = P_YEAR
 14          GROUP BY EXTRACT(MONTH FROM ORDER_DATE)
 15          ORDER BY MONTH_NO
 16      )
 17      LOOP
 18          DBMS_OUTPUT.PUT_LINE(
 19              'Month: ' || R.MONTH_NO ||
 20              ' Total Orders: ' || R.TOTAL_ORDERS ||
 21              ' Revenue: ' || R.TOTAL_REVENUE
 22          );
 23      END LOOP;
 24  END;
 25  /

Procedure created.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC MONTHLY_REVENUE_REPORT(2026);
MONTHLY REVENUE REPORT
---------------------------
Month: 9 Total Orders: 5 Revenue: 106200
Month: 11 Total Orders: 3 Revenue: 161600

PL/SQL procedure successfully completed.

SQL> SHOW ERRORS PROCEDURE MONTHLY_REVENUE_REPORT;
No errors.
```

- Customer purchase report.

```
SQL> CREATE OR REPLACE PROCEDURE CUSTOMER_PURCHASE_REPORT
  2  AS
  3  BEGIN
  4      DBMS_OUTPUT.PUT_LINE('CUSTOMER PURCHASE REPORT');
  5      DBMS_OUTPUT.PUT_LINE('--------------------------------');
  6
  7      FOR R IN (
  8          SELECT C.CUSTOMER_ID,
  9                 C.CUSTOMER_NAME,
 10                 COUNT(O.ORDER_ID) AS TOTAL_ORDERS,
 11                 NVL(SUM(O.TOTAL_AMOUNT), 0) AS TOTAL_PURCHASE
 12          FROM CUSTOMER C
 13          LEFT JOIN ORDERS O
 14          ON C.CUSTOMER_ID = O.CUSTOMER_ID
 15          GROUP BY C.CUSTOMER_ID, C.CUSTOMER_NAME
 16          ORDER BY C.CUSTOMER_ID
 17      )
 18      LOOP
 19          DBMS_OUTPUT.PUT_LINE(
 20              'Customer: ' || R.CUSTOMER_NAME ||
 21              ' | Orders: ' || R.TOTAL_ORDERS ||
 22              ' | Purchase Amount: ' || R.TOTAL_PURCHASE
 23          );
 24      END LOOP;
 25  END;
 26  /

Procedure created.

SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC CUSTOMER_PURCHASE_REPORT;
CUSTOMER PURCHASE REPORT
--------------------------------
Customer: Arun | Orders: 3 | Purchase Amount: 227500
Customer: Priya | Orders: 2 | Purchase Amount: 26600
Customer: Kumar | Orders: 1 | Purchase Amount: 1500
Customer: Divya | Orders: 1 | Purchase Amount: 4200
Customer: Rahul | Orders: 1 | Purchase Amount: 8000
Customer: Rohan | Orders: 0 | Purchase Amount: 0

PL/SQL procedure successfully completed.

SQL> SHOW ERRORS PROCEDURE CUSTOMER_PURCHASE_REPORT;
No errors.
```

5. Test Database Objects Tasks:

Students should test:

Views

Check:

- Correct data retrieval.
- Report accuracy.

```
SQL> SELECT * FROM EINFO;

       EID NAME
---------- --------------------
      1002 Biju
      1005 Sam
      1010 Krishnakumari

SQL> SELECT * FROM CUSTOMER_ORDER_REPORT;

CUSTOMER_NAME                                      ORDER_ID ORDER_DAT
-------------------------------------------------- -------- --------
TOTAL_AMOUNT ORDER_STAT
------------ ----------
Arun                                                   1006 07-NOV-26
      135000 Pending

Arun                                                   1007 07-NOV-26
       25000 Shipped

Arun                                                   1001 27-SEP-26
       67500 Delivered


CUSTOMER_NAME                                      ORDER_ID ORDER_DAT
-------------------------------------------------- -------- --------
TOTAL_AMOUNT ORDER_STAT
------------ ----------
Priya                                                  1008 07-NOV-26
        1600 Shipped

Priya                                                  1002 27-SEP-26
       25000 Shipped

Kumar                                                  1003 27-SEP-26
        1500 Delivered


CUSTOMER_NAME                                      ORDER_ID ORDER_DAT
-------------------------------------------------- -------- --------
TOTAL_AMOUNT ORDER_STAT
------------ ----------
Divya                                                  1004 27-SEP-26
        4200 Pending

Rahul                                                  1005 27-SEP-26
        8000 Delivered


8 rows selected.
```

```
SQL> SELECT * FROM PRODUCT_SALES_REPORT;

PRODUCT_NAME         QUANTITY_SOLD TOTAL_REVENUE
-------------------- ------------- -------------
Laptops                          3        202500
Mobile                           2         50000
TShirt                           4          3100
Shoes                            2          4200
Keyboard                         1          8000

SQL> SELECT * FROM PAYMENT_STATUS_REPORT;

  ORDER_ID PAYMENT_MODE         PAYMENT_ST TRANSACTION_AMOUNT
---------- -------------------- ---------- ------------------
      1007 UPI                  Success                25000
      1008 UPI                  Success                 1600
      1001 UPI                  Success                67500
      1002 Card                 Success                25000
      1003 Cash                 Success                 1500
      1004 Net Banking          Pending                 4200
      1005 UPI                  Success                 8000
      1001 Card                 Failed                  1000
      1002 UPI                  Pending                 5000
      1003 Net Banking          Failed                  2000

10 rows selected.

SQL> SELECT SUM(TOTAL_AMOUNT)
  2  FROM ORDERS;

SUM(TOTAL_AMOUNT)
-----------------
           267800
```

Procedures

Test:

- Procedure execution.
- Input parameters.
- Output results.

```
SQL> SET SERVEROUTPUT ON;
SQL>
SQL> EXEC DAILY_SALES_REPORT(DATE '2026-11-07');
DAILY SALES REPORT
---------------------------
Order ID: 1006 Amount: 135000 Status: Pending
Order ID: 1007 Amount: 25000 Status: Shipped
Order ID: 1008 Amount: 1600 Status: Shipped
Total Orders: 3
Total Sales: 161600

PL/SQL procedure successfully completed.

SQL> EXEC MONTHLY_REVENUE_REPORT(2026);
MONTHLY REVENUE REPORT
---------------------------
Month: 9 Total Orders: 5 Revenue: 106200
Month: 11 Total Orders: 3 Revenue: 161600

PL/SQL procedure successfully completed.

SQL> EXEC CUSTOMER_PURCHASE_REPORT;
CUSTOMER PURCHASE REPORT
------------------------------
Customer: Arun | Orders: 3 | Purchase Amount: 227500
Customer: Priya | Orders: 2 | Purchase Amount: 26600
Customer: Kumar | Orders: 1 | Purchase Amount: 1500
Customer: Divya | Orders: 1 | Purchase Amount: 4200
Customer: Rahul | Orders: 1 | Purchase Amount: 8000
Customer: Rohan | Orders: 0 | Purchase Amount: 0

PL/SQL procedure successfully completed.
```

```
SQL> SELECT OBJECT_NAME, STATUS
  2  FROM USER_OBJECTS
  3  WHERE OBJECT_TYPE = 'PROCEDURE'
  4  AND OBJECT_NAME IN (
  5      'DAILY_SALES_REPORT',
  6      'MONTHLY_REVENUE_REPORT',
  7      'CUSTOMER_PURCHASE_REPORT'
  8  );

OBJECT_NAME
-----------------------------------
STATUS
-------
CUSTOMER_PURCHASE_REPORT
VALID

DAILY_SALES_REPORT
VALID

MONTHLY_REVENUE_REPORT
VALID
```

Functions

Verify:

- Calculation accuracy.
- Return values

```
SQL> SELECT OBJECT_NAME
  2  FROM USER_OBJECTS
  3  WHERE OBJECT_TYPE = 'FUNCTION';

OBJECT_NAME
--------------------------------------------------------------------------------
YEARLY_SALARY
CALCULATE_TOTAL_SALES
CALCULATE_CUSTOMER_SPENDING
CALCULATE_PRODUCT_REVENUE

SQL> SELECT Calculate_Total_Sales(
  2  TO_DATE('01-11-2026','DD-MM-YYYY'),
  3  TO_DATE('30-11-2026','DD-MM-YYYY')
  4  ) FROM DUAL;

CALCULATE_TOTAL_SALES(TO_DATE('01-11-2026','DD-MM-YYYY'),TO_DATE('30-11-2026','D
--------------------------------------------------------------------------------
                                                                         161600

SQL>
SQL> SELECT Calculate_Customer_Spending(1001)
  2  FROM DUAL;

CALCULATE_CUSTOMER_SPENDING(1001)
---------------------------------
                           227500

SQL>
SQL> SELECT Calculate_Product_Revenue('P1001')
  2  FROM DUAL;

CALCULATE_PRODUCT_REVENUE('P1001')
----------------------------------
                            202500

SQL> CREATE OR REPLACE VIEW LOW_STOCK_VIEW AS
  2  SELECT PRODUCT_ID, PRODUCT_NAME, STOCK_QUANTITY
  3  FROM PRODUCT
  4  WHERE STOCK_QUANTITY <= 5;

View created.

SQL> SELECT * FROM LOW_STOCK_VIEW;

PRODUCT_ID PRODUCT_NAME         STOCK_QUANTITY
---------- -------------------- --------------
P1001      Laptops                           0
```